Algoritmos de Busca

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Busca em Tabelas

- Algoritmos fundamentais: busca em tabelas estão entre os mais usados
- Exemplo: um sistema de banco de dados
  - As operações de busca e recuperação dos dados são bastante frequentes
  - Considere também os programas “buscadores” da Internet (Google, Yahoo, etc.). Em todos eles são usados de alguma maneira os algoritmos básicos de busca em tabelas ou arquivos
- Vamos estudar os métodos internos de busca, isto é, os dados estão em tabelas na memória
  - No caso de arquivos em dispositivos externos (discos, memórias de massa, etc.) há outras variáveis a se considerar
  - O tempo de acesso pode envolver operações mecânicas (movimento do braço e rotação no caso do disco) que podem em ser muito mais significativos que o tempo gasto pelo algoritmo
  - Mas de maneira geral, se um algoritmo é bom para se efetuar busca em tabelas na memória, sempre pode ser adaptado para funcionar também com arquivos externos

Busca em Tabelas

- Python: tabelas são representadas por lista (list)
  - Já possui funções intrínsecas: count e index - procurar e localizar elementos em uma tabela ou lista
- **lst.count(x)** – retorna quantas vezes o elemento x ocorre na lista **lst**
- **lst.index(x)** – retorna o índice da primeira ocorrência de x na lista **lst** (se não existe x retorna ValueError)
- **lst.index(x, i)** – retorna o índice da primeira ocorrência de x a partir de **lst[i]**
- **lst.index(x, i, f)** – retorna o índice da primeira ocorrência de x a partir de **lst[i]** até **lst[f-1]**

Busca em Tabelas

- Python possui ainda o operador **in:** verifica se elemento está ou não numa tabela

  if x **in** lst: . . .

  if x not **in** lst: . . .

  while x **in** list: . . .

- **Nosso objetivo:** estudar os algoritmos de busca em tabelas e entender como essas funções e operadores estão implementados
  - Percorrer a tabela de alguma forma, procurando o elemento
- Vamos verificar com mais profundidade os algoritmos que fazem a busca, ou os algoritmos usados nestas funções

Busca Sequencial

- Varrer uma tabela a procura de um determinado elemento
  - Verifica-se ao final se o elemento foi ou não foi encontrado
- A função busca sequencial procura elemento igual a *x* num vetor *a* de *n* elementos, devolvendo
  - –1 se não encontrou
  - o índice do primeiro elemento igual a *x* no vetor
  - (É claro que pode haver mais de um elemento igual a *x*)

Busca Sequencial

```python
def busca_sequencial(a, x):
    n = len(a)
    for i in range(n):
        if a[i] == x: return i
    # foi até o fim e não achou
    return -1
```

```python
def busca_sequencial_alternativa(a, x):
    n = len(a)
    i = 0
    while i < n and a[i] != x:
        i += 1
    # verifica se parou porque chegou ao fim ou encontrou um igual
    if i == n: return -1 # chegou a fim sem encontrar
    return i # encontrou um igual
```

Análise da Busca Sequencial

- Considerando a função busca_sequencial na primeira versão:
  - Quantas vezes a comparação (a[i] == x) é executada?
- máximo = n (quando não encontra ou quando x = a[n-1])
- mínimo = 1 (quando x = a[0])
- médio = (n+1)/2
- *Há uma hipótese importante considerada para se chegar ao número médio de (n+1)/2. Temos que supor que a quantidade de comparações pode ser 1, 2, 3,..., n com a mesma probabilidade* - **lembrando, o caso médio não é a média entre pior e melhor caso!!!**

Análise da Busca Sequencial

- O algoritmo é SEMPRE **O(n)**
- Assim, esse algoritmo destina-se a buscas com *n* pequeno
  - Quando n é grande, desempenho ineficiente
  - Para uma tabela com 1kk elementos -> se cada repetição demorar cerca de 100 microssegundos -> o tempo é quase 2 minutos
- Melhoria para a busca sequencial: quando existem elementos que podem ocorrer com maior frequência
  - Colocá-los no início da tabela -> serão encontrados com menos comparações (como implementar? Qual o custo?)

Análise da Busca Sequencial

- Supondo que a probabilidade do elemento procurado ser a[i] é *pi*:
  - Σ pi (0<=i<n) é a probabilidade de x estar na tabela
  - Σ pi < 1 se existe alguma chance de x não estar na tabela
  - 1 - Σ pi é a probabilidade de x não estar na tabela
- O número médio de comparações será então:
  - 1.p0 + 2.p1 + 3.p2 + ... + n.p(n-1) + n.(1 - Σ pi)
- Se os elementos mais procurados estão no início da tabela, o número médio de comparações é menor (fórmula acima) - desigualdade de rearranjo (maiores pesos e menores valores) - elementos de maior chance nas primeiras posições

Busca Sequencial em tabela ordenada

- Quando a tabela está ordenada uma melhoria pode ser feita:
  - Se encontrarmos um elemento que já é maior que x -> encerra busca pois todos os demais serão maiores

```
def busca_sequencial(a, x):
    n = len(a)
    for i in range(n):
        if a[i] == x: return i  # encontrou
        if a[i] > x: return -1
    # foi até o final e não encontrou
    return -1
```

```
def busca(a, x):
    n = len(a)
    i = 0
    while i < n and a[i] < x: i += 1
    # verifica se parou porque chegou ao fim ou encontrou
    um igual
    if i == n: return -1    # chegou ao final
    if a[i] == x: return i  # achou
    return -1  # encontrou um maior então não adianta
                procurar mais
```

Busca Binária

- A busca binária assemelha-se a busca de palavras no dicionário.
  - Não vamos verificando folha a folha (busca sequencial)
  - Abrimos o dicionário mais ou menos no meio e daí só há três possibilidades

    a) A palavra procurada está na página aberta

    b) A palavra está na parte esquerda do dicionário

    c) A palavra está na parte direita do dicionário

- Caso não ocorra a situação (a), repetimos o mesmo processo com as páginas da parte direita ou da parte esquerda, onde a palavra tem chance de ser encontrada
- Em cada comparação eliminamos cerca de metade das páginas do dicionário

Busca Binária

- Algoritmicamente, temos:

  a) Testa com o elemento do meio da tabela;

  b) Se for igual, termina o algoritmo porque encontrou o elemento;

  c) Se o elemento do meio é maior, repete o processo considerando a tabela do inicio até o meio – 1;

  d) Se o elemento do meio é menor, repete o processo considerando a tabela do meio + 1 até o final;

- Se o elemento não está na tabela, chegará o momento em que a tabela restante terá zero elementos que é o outro critério de parada do algoritmo.
- Observe que se ocorreu a situação (c) ou (d), a tabela foi reduzida à metade.

Busca Binária (Iterativa)

```
def buscabinaria(a, x):
    n = len(a)
    inicio = 0
    final = n - 1
    # Enquanto a tabela tem elementos continue procurando
    while inicio <= final:
        meio = (inicio + final) // 2
        if a[meio] == x: return meio # encontrou o procurado
        # verifica se continua a procura na parte superior ou inferior
        if x < a[meio]: final = meio -1 # parte superior
        else: inicio = meio + 1         # parte inferior
    # Se chegou aqui é porque esgotou as possibilidades e não encontrou
    return -1
```

Busca Binária Alternativa

- Outra forma de se pensar o algoritmo de busca binária:
  - Primeiro elemento (base) e um tamanho (N)
  - A cada iteração compara-se o elemento procurado com o elemento médio da tabela (x == a[base + N // 2]?)
  - Se encontrar -> termina
  - Se x > a[base + N // 2] -> o elemento procurado deve estar acima e a nova base fica sendo (base + N // 2 + 1) enquanto
    - o novo valor de N fica em (N // 2) se N é ímpar
    - ou (N // 2 - 1) se N é par
  - Se x < a[base + N // 2] -> o elemento procurado deve estar abaixo e a base permanece a mesma enquanto N fica em (N // 2)
  - Inicialmente, a base é zero e o algoritmo termina quando N fica zero -> a tabela terminou e não encontramos o elemento procurado

Busca Binária Alternativa Implementação

```
# Procura X em L[base], ..., L[base + N - 1]
# Busca binária em tabela ordenada - versão não recursiva
def BBNR(L, X, base, N):
    # Continua a busca enquanto N > 0
    while N > 0:
        # meio da tabela
        meio = base + N // 2
        if L[meio] == X: return meio
        # Cálculo do tamanho da parte de cima e da parte de
baixo
        if N % 2 == 0:
            # N é par
            tam_inf, tam_sup = N // 2, N // 2 - 1
        else:
            # N é impar
            tam_inf, tam_sup = N // 2, N // 2
        # Continua a busca na parte de cima ou de baixo.
        if X > L[meio]:
            base, N = meio + 1, tam_sup
        else:
            N = tam_inf
    # Se saiu do while - não encontrou
    return -1
```

Busca Binária (Recursiva)

- A cada repetição, realizam-se as mesmas operações numa tabela menor (metade)
- Verifica-se o elemento médio da tabela
  - Se for o buscado -> termina
  - Se for maior -> executa-se a busca binária na metade superior
  - Senão -> executa-se a busca binária na metade inferior
  - A busca termina quando a tabela tiver zero elementos
- Observe que a tabela superior vai do elemento médio mais 1 até o final e a tabela inferior vai do início até o elemento médio menos 1

Busca Binária (Recursiva) - Implementação

```
def BBR(a, inicio, final, x):
    if inicio > final: return -1 # não encontrou
    meio = (inicio + final) // 2
    if a[meio] == x: return meio  # encontrou
    # procura na parte superior ou inferior da tabela
    if a[meio] > x: return BBR(a, inicio, meio - 1, x)
    return BBR(a, meio + 1, final, x)
```

Busca Binária Alternativa (Recursiva)

```
# Procura X em L[base], ..., L[base + N - 1]
# Busca binária em tabela ordenada - versão recursiva
def BBR(L, X, base, N):
   if N == 0: return -1 # terminou a busca
   # meio da tabela
   meio = base + N // 2
   if L[meio] == X: return meio
   # Cálculo do tamanho da parte de cima e da parte de baixo
   if N % 2 == 0:
       # N é par
       tam_inf, tam_sup = N // 2, N // 2 - 1
   else:
       # N é impar
       tam_inf, tam_sup = N // 2, N // 2
   # Continua a busca na parte de cima ou de baixo.
   if X > L[meio]: return BBR(L, X, meio + 1, tam_sup)
   return BBR(L, X, Base, tam_inf)
```

Busca Binária com sub-listas (Recursiva)

- Neste caso perde-se a informação do índice na tabela em que está o elemento
- Há um problema de uso de memória excessivo porque a cada nova chamada, novas listas são criadas

```
def BBsub(a, x):
    n = len(a)
    if n == 0: return False
    meio = (n - 1) // 2
    if x == a[meio]: return True
    if x > a[meio]: return BBsub(a[meio+1:], x)
    return BBsub(a[:meio], x)
```

Informações expostas pela busca binária

- Podemos extrair mais informações do algoritmo de busca binária, além do resultado se achou ou não achou
  - Quando não encontramos o elemento, a busca termina, nas proximidades de onde o elemento deveria estar
  - Pode ser útil para encontrar onde inserir um elemento na tabela

i	0	1	2	3	4	5	6	7	8
A[i]	12	15	18	18	18	20	25	38	44

- De uma forma geral, se **busca_binariaP(x, a, n)** devolve k, então de a[0] até a[k-1] todos são menores que x e de a[k] até a[n-1] todos são maiores ou iguais a x

Informações expostas pela busca binária

- Esta versão tem duas aplicações interessantes:
  - Se é necessário inserir um elemento que ainda não está na tabela, a busca devolve exatamente o lugar em que ele deverá ser inserido -> exige o deslocamento de todos acima dele para abrir espaço
  - Quando há elementos repetidos na tabela, a busca devolve exatamente o primeiro igual -> todos os outros iguais estarão após ele

Informações expostas pela busca binária

```
# procura x em a[0] até a[n-1] e devolve:
#   0 se x<= a[0]
#   n se x > a[n-1]
#   R se a[R-1] < x <= a[R]
def binariaP(x, a):
   n = len(a)
   # se x está fora da tabela
   if x <= a[0]: return 0
   if x > a[n-1]: return n
   # x pode estar dentro da tabela
   L = 0
   R = n-1
   while R - L > 1:
       M = (R + L) // 2
       if x <= a[M]: R = M
       else: L = M
   return R

k = binariaP(z, b)
m = len(b)
if k < m and b[k] == z: # achou
else: # não achou
```

Análise da Busca Binária

- A comparação a[meio] == x é o comando significativo para a análise do tempo de execução -> representa a quantidade de repetições que serão feitas até o final do algoritmo
  - Proporcional à quantidade de repetições (comando while)

- Quantas vezes a comparação a[meio] == x é efetuada?
  - Mínimo = 1 (encontra na primeira)
  - Máximo = ?
  - Médio = ?

Análise da Busca Binária - Máximo de Comparações

- A cada iteração a tabela fica dividida ao meio
- A nova busca é feita numa tabela que tem no máximo a metade dos elementos
- Enquanto não se encontra o elemento procurado, as comparações continuam até o valor k (iterações) tal que:
  - $2^k \leq N < 2^{(k+1)}$
  - Ou $k \leq \lg N < k + 1$ ($\lg N$ é o log de N na base 2)
  - Ou ainda (somando – k – $\lg N$ em todos os membros): $-\lg N \leq -k < -\lg N + 1$
  - Ou (multiplicando por -1) $\lg N - 1 < k \leq \lg N$
  - Como estamos interessados no valor $(k + 1)$ termos: $\lg N < (k + 1) \leq 1 + \lg N$
  - Assim o número máximo de comparações está entre $\lg N$ e $1 + \lg N$
- Assim, o algoritmo é **O(lg N)** , ou **O(log N)**!!!
  - Uma tabela de 1.000.000 de elementos -> $\lg (1.000.001) = 20$; busca sequencial: 1.000.000

Análise da Busca Binária

N	lg (N+1)
100	7
1.000	10
10.000	14
100.000	17
1.000.000	20
10.000.000	24
100.000.000	27
1.000.000.000	30

Análise da Busca Binária - Média de Comparações

- Será que o número médio é muito diferente da média entre o máximo e o mínimo?
- Supondo que sempre encontramos o elemento procurado
  - Note que quando não encontramos o elemento -> número de comparações é igual ao máximo
  - Assim, no caso geral, a média estará entre o máximo e a média supondo que sempre vamos encontrar o elemento
- Supondo que temos N elementos e que a probabilidade de procurar cada um é sempre 1/N.
- Vamos considerar N = 2^k - 1 - quebra do vetor nas metades (elems)
- 1 comparação na tabela de N elementos, 2 comparações em 2 tabelas de N/2 elementos, 3 comparações em 4 tabelas de N/4 elementos, 4 comparações em 8 tabelas de N/8 elementos, e assim por diante:
  - (por indução) ~ log N - 1 + (N+1)/N ≈ log N (próximo do máximo)

Árvores de Busca Binária

- Pior caso: o número de comparações é igual ao número de nós da árvore -> no caso em que a árvore tem tantos níveis quanto o número de elementos
  - Complexidade é O(N) -> árvore degenerada em lista encadeada
- A complexidade de busca é a altura da árvore: é conveniente que a árvore tenha sempre altura mínima -> esteja balanceada ou parcialmente balanceada
- A árvore que possui tal propriedade é uma AB dita **completa** (todos os nós com filhos vazios estão no último ou penúltimo nível)
  - Complexidade é O(log N) -> se T é uma AB completa com N>0 nós então T possui altura H mínima e H=1+log N (considerando o valor de log N truncado) (para raíz com altura 1)

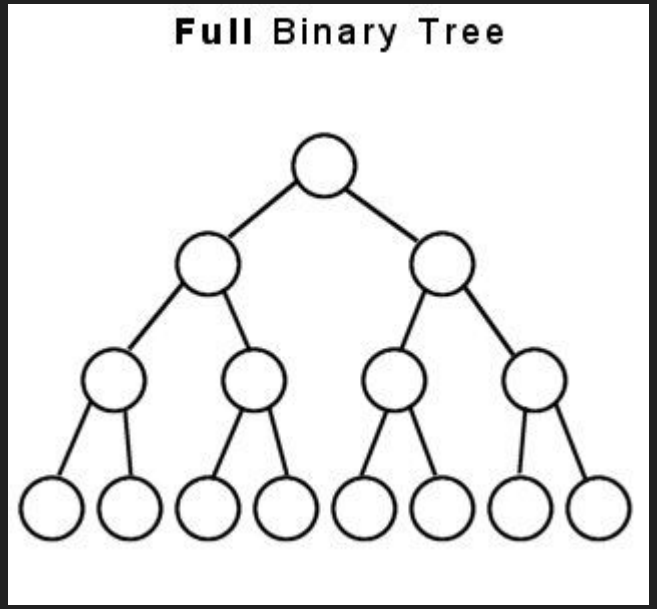
Árvores de Busca Binária

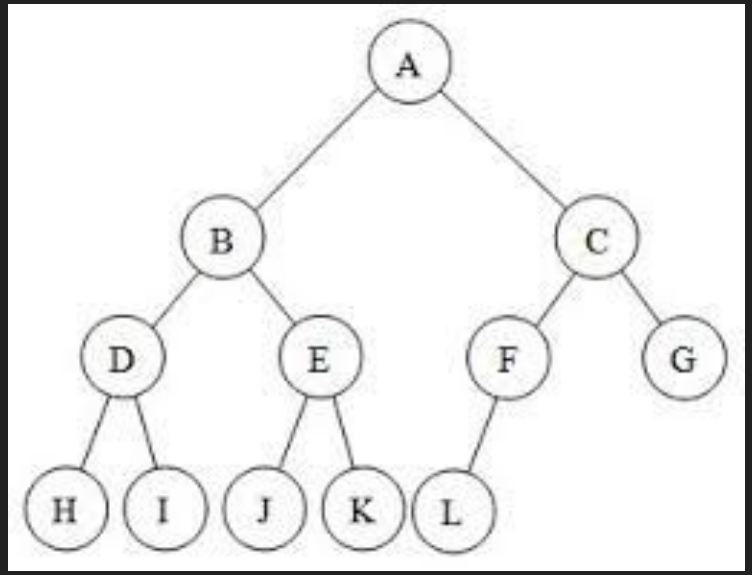
- Seja T uma AB completa com N nós e altura H
  - Então $2^{(H-1)} \leq N \leq 2^H - 1$
- Se a AB completa possui apenas 1 nó no seu nível inferior então $N = 2^{(H-1)}$
- Se a AB completa está cheia $N = 2^H - 1$

```
# Procura elemento com info igual a v na ABB h
# Versão recursiva
def busca(h, v):
   if h is None: return None
   t = h._info
   if t == v: return h
   if v < t:
       return busca(h._eprox, v) # procura à esquerda
   else:
       return busca(h._dprox, v) # procura à direita
```

Árvore Binária Completa

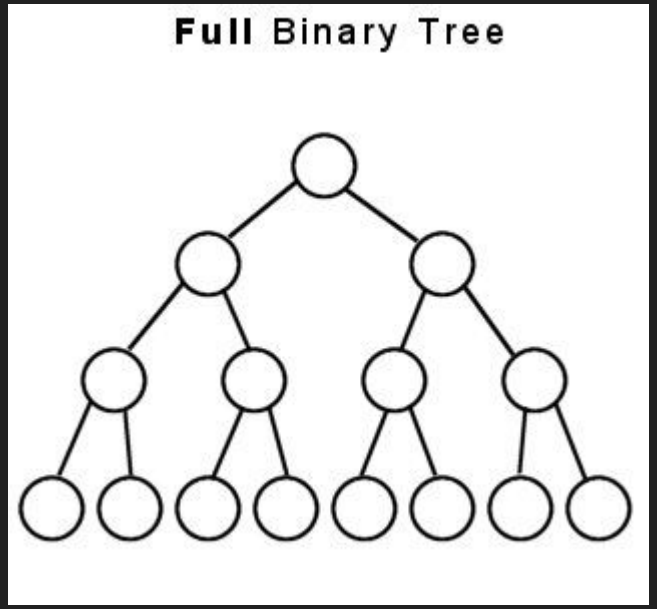
- h = ?

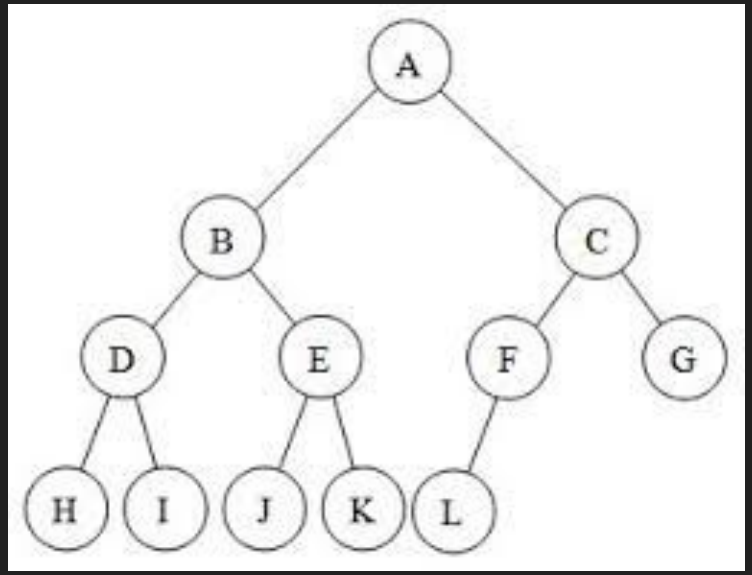




Árvore Binária Completa

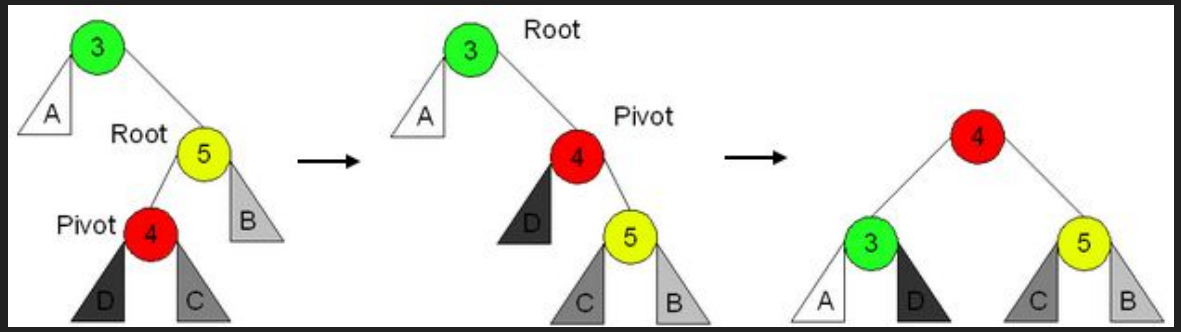
- **h = log N + 1 ~ O(log N) - exatamente o mesmo comportamento da BB**





Árvores Balanceadas (AVL e Rubro-Negras)

- **Qual é a complexidade de busca para estas árvores?**



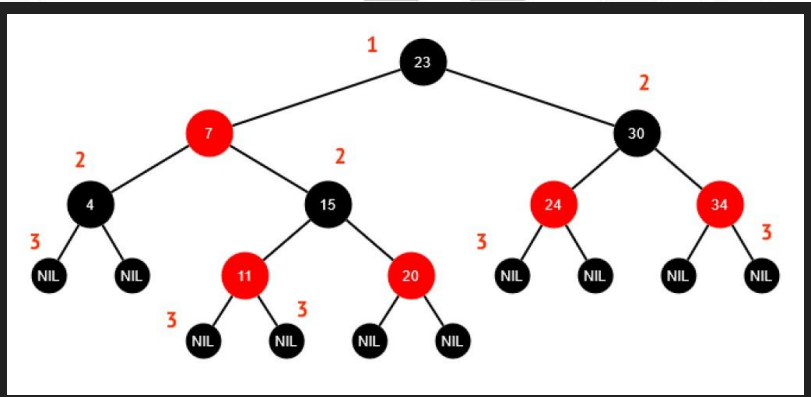


Tabela Hash

Impacto da função hash:

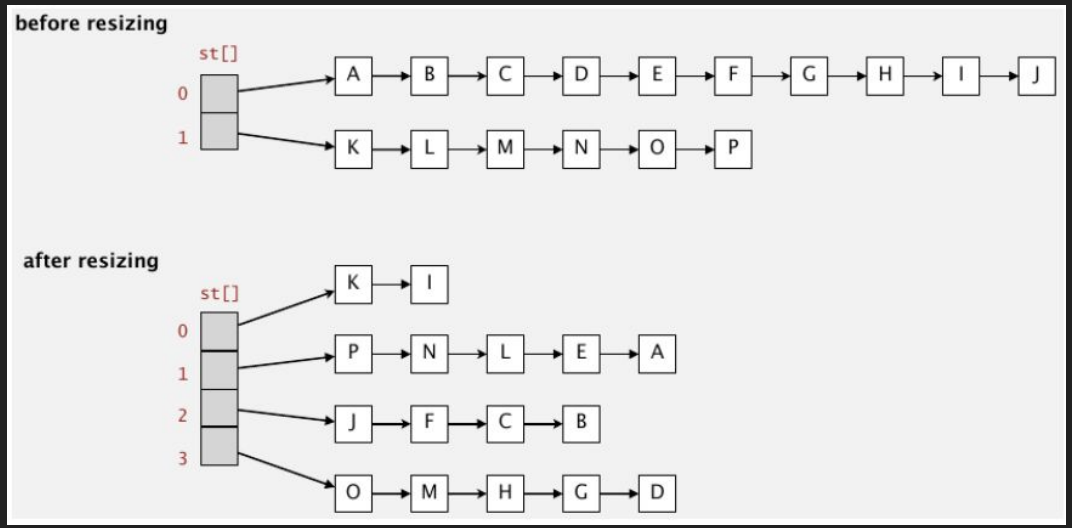
- Função hash deve ser determinística. Para uma determinada chave a função sempre retorna o mesmo valor de hash
- A função de hash deve sempre retornar um valor de hash dentro dos limites da tabela [0,M−1] , onde M é o tamanho da tabela
- Uniforme: todos os índices da tabela devem ter aproximadamente a mesma chance de serem mapeados pela função hash
    - Importante para distribuir os elementos uniformemente pela tabela -> impacta diretamente nas colisões
- A função de hash deve ser executada em tempo constante O(1) (ou muito próximo disso)

Separate Chaining - Análise

- Proposição: assumindo hashing uniforme, a probabilidade que o número de chaves em uma lista (*bucket*) seja ~ c(N/M) e muito próxima de 1
  - Tamanhos das listas seguem uma distribuição binomial -> número de *probes* para busca/inserção é proporcional a N/M. (M vezes mais rápido que busca linear.)
- M muito grande: desperdício de memória.
- M muito pequeno: cadeias (listas) muito longas
- Escolha típica: M ~ N/4 ⇒ ops em tempo constante

Separate Chaining - Análise

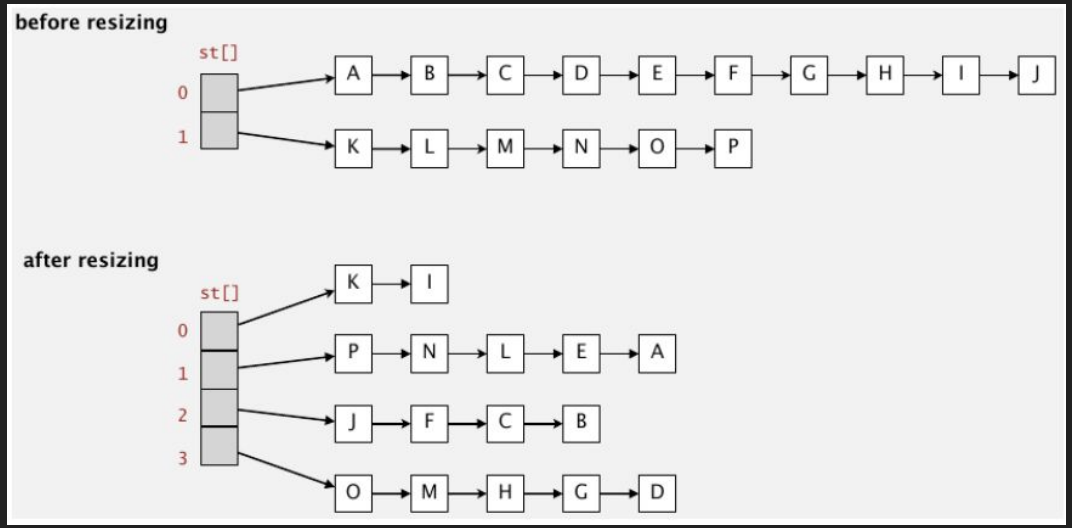
- Objetivo: Manter o comprimento médio das listas constante.
  - Quando N/M ≥ 8: aumentar (duplicar?) a tabela?
  - Quando N/M ≤ 2: diminuir a tabela?
    - Todo ajuste requer um rehash de todas as chaves!!!



Separate Chaining - Análise

	search	insert	delete	search hit	insert	delete
separate chaining	N	N	N	3-5 *	3-5 *	3-5 *

* under uniform hashing assumption



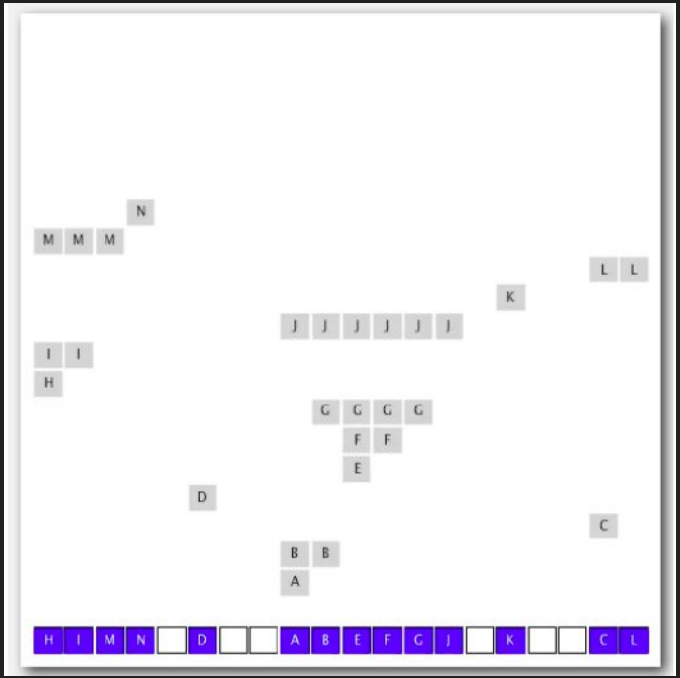
Linear Probing

- Hash: mapear a chave em um inteiro i entre 0 e M − 1.
- Inserção : insira no índice i se estiver livre, senão tente i + 1, i + 2, etc.
- Busca: busque no array no índice i, se estiver ocupado mas nao bater a chave, tente i + 1, i + 2, etc.
- Nota: tamanho M da tabela precisa ser maior que o número de chaves N.



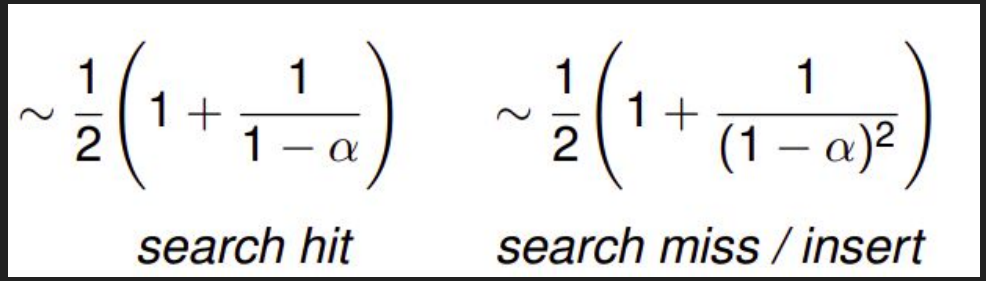
Linear Probing - Clustering

- Cluster: um bloco contíguo de chaves.
- Observação: novas chaves tendem a cair no meio de clusters.



Linear Probing - Análise

- Proposição : sob a suposição de hashing uniforme, o número médio de probes em uma tabela de tamanho M que contém N = αM chaves (α < 1) é:



- Escolha típica: α = N/M ~ 1/2 ⇒ Hit: 3/2 – Miss: 5/2

Linear Probing - Análise

- Objetivo: Manter a taxa de ocupação da tabela em N/M ≤ 1/2.
  - Quando N/M ≥ 1/2: aumentar a estrutura
  - Quando N/M ≤ 1/8: diminuir a estrutura
    - Todo ajuste requer um rehash de todas as chaves
- Ponto crítico: remoção!



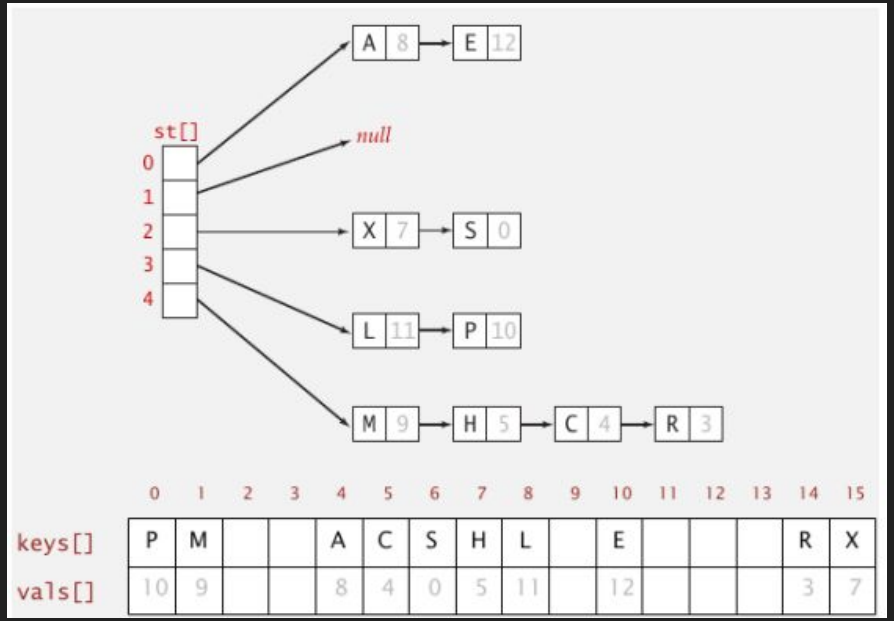
Linear Probing - Análise

	search	insert	delete	search hit	insert	delete
linear probing	N	N	N	3-5 *	3-5 *	3-5 *



Separate Chaining *vs.* Linear Probing

- Separate chaining:
  - Desempenho degrada graciosamente.
  - Clustering menos sensível a uma função hash ruim
- Linear probing:
  - Menos espaço desperdiçado.
  - Melhor uso de cache.





Dúvidas?



Obrigado!

Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr
- https://br.linkedin.com/in/luis-souza-22829368